

# Reviewer Pack — Symmetric Tensor Rank Gap in Permanent vs Determinant Decomp...

Entry 8edb074fa2a1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 22:28:35 UTC

## Symmetric Tensor Rank Gap in Permanent vs Determinant Decompositions

**Entry ID:** 8edb074fa2a1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 22:28:35 UTC

### 1. Conjecture

**Field A** (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Boolean Circuit Complexity (Permanent vs Determinant)

#### Statement:

For a homogeneous polynomial  $f$  of degree  $n$ , let  $\mu(f)$  denote the multiplicity of the irreducible representation indexed by  $(n-1,1)$  in the decomposition of  $\text{Sym}^n(f)$ . Then  $\mu(\text{perm}_n) \geq 2^{\Omega(n)}$  while  $\mu(\text{det}_m) \leq 2^{O(\sqrt{n})}$  for all  $m < n^{1.5}$ .

#### Rationale (proposer's reasoning):

The  $(n-1,1)$  partition corresponds to the standard representation of the symmetric group, which captures the alternating nature of the determinant. The permanent's symmetric structure forces higher multiplicities in this decomposition, while the determinant's anti-symmetry suppresses it. This gap could expose algebraic structure in circuit complexity.

**Taxonomy category:** GCT\_DET\_PERM (status at proposal time: partially\_alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** a08160299055ec2c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.95       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

| Barrier       | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|---------------|---------|------------|------------------|------------------|
| ALGEBRIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON   | SAFE    | 0.95       | UNCERTAIN        | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        factor = Fraction(matrix[i][i])
        for j in range(n):
            matrix[i][j] /= factor
        for j in range(n):
            if i != j:
                factor = Fraction(matrix[j][i])
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = Fraction(0)
    sign = 1
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += sign * matrix[0][j] * determinant(submatrix)
```

```

        sign *= -1
    return det

def map_clause_to_tensor(clause, n):
    tensor = [[0] * n for _ in range(n)]
    for var in clause:
        if var > 0:
            tensor[var-1][var-1] += 1
        else:
            tensor[-var-1][-var-1] += 1
    return tensor

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    perm_tensor = [[0] * n for _ in range(n)]
    det_tensors = []
    for i in range(n):
        det_tensors.append([[0] * n for _ in range(n)])

    # Map permanent to tensor
    for i in range(n):
        perm_tensor[i][i] += 1

    # Map determinant-like tensors
    for m in range(1, int(n**1.5) + 1):
        det_tensors[m-1][0][m-1] += 1
        det_tensors[m-1][m-1][0] += 1

    # Compute symmetric tensor powers
    perm_power = perm_tensor
    for _ in range(2, n+1):
        perm_power = matrix_multiply(perm_power, perm_tensor)

    det_powers = []
    for m in range(1, int(n**1.5) + 1):
        det_power = det_tensors[m-1]
        for _ in range(2, n+1):
            det_power = matrix_multiply(det_power, det_tensors[m-1])
        det_powers.append(det_power)

    # Decompose into irreducible representations using Young tableaux counting
    def count_young_tableaux(n, k):
        if n == 0 or k == 0:
            return 1
        return (n + k - 1) * count_young_tableaux(n-1, k-1) // k

    perm_multiplicity = count_young_tableaux(n, n-1)
    det_multiplicities = [count_young_tableaux(m, m-1) for m in range(1, int(n**1.5) + 1)]

    # Measure  $\mu(f)$  for both permanent and determinant-like tensors
    perm_mu = perm_multiplicity
    det_mus = det_multiplicities

    # Check if  $\mu(\text{perm}_n)$  exceeds  $\mu(\text{det}_m)$  by exponential factors
    conjecture_holds = all(perm_mu >= 2**(m * math.log(n, 2)) for m in range(1, int(n**1.5) + 1))

```

```

counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "mu",
    "metric_value": perm_mu,
    "instances_tested": n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30*10**4, 10**4))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_mu = sum(r["metric_value"] for r in results) / len(results)
    std_mu = math.sqrt(sum((r["metric_value"] - mean_mu)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_mu} std={std_mu} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1bf997b9.py", line 125, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1bf997b9.py", line 79, in run_trial
    det_tensors[m-1][0][m-1] += 1
    ~~~~~^~~~~~
IndexError: list index out of range

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed with IndexError before producing results, preventing validation of conjecture | next: Fix the list index out-of-range error in test\_1bf997b9.py and re-run with proper parameter bounds

## 11. Audit log (LLM calls)

**Total LLM calls:** 8

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 111615       |
| 2 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 78905        |
| 3 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 24164        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20741        |
| 5 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 18637        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15716        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14024        |
| 8 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 15495        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 299297 ms total latency. Provider mix: {'ollama\_remote': 8}

(full prompt+response transcripts available in research/audit/8edb074fa2a1.jsonl)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8edb074fa2a1.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/8edb074fa2a1.tar.gz (if generated)